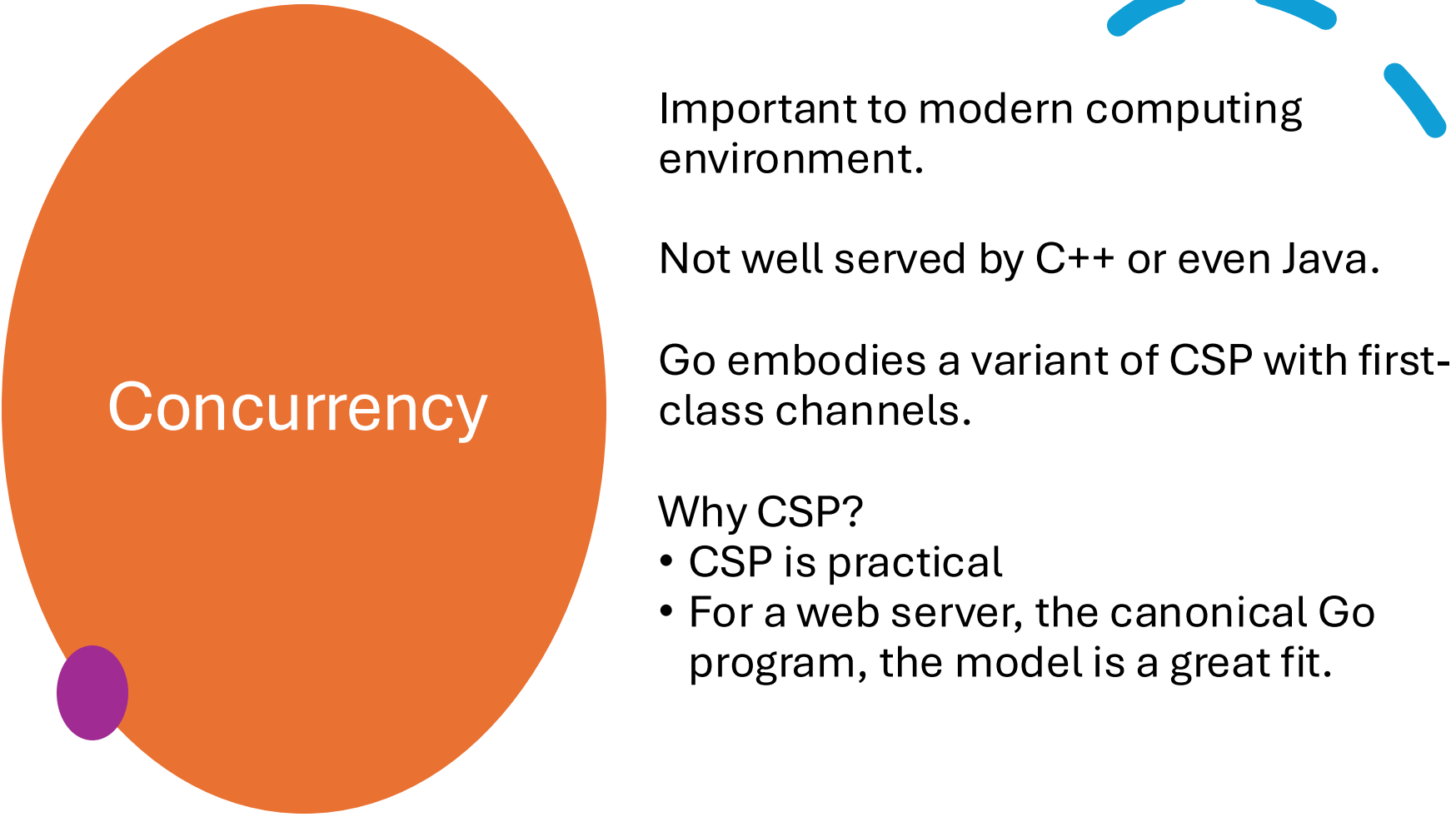


Concurrency in Go





Concurrency

Important to modern computing environment.

Not well served by C++ or even Java.

Go embodies a variant of CSP with first-class channels.

Why CSP?

- CSP is practical
- For a web server, the canonical Go program, the model is a great fit.

Experience shows this is a practical design

Concurrency

Go enables

- simple, safe concurrent programming.
- It doesn't forbid bad programming.
- Focus on composition of regular code.

Caveat

- not purely memory safe; sharing is legal.
- Passing a pointer over a channel is idiomatic.

.

Experience shows this is a practical design

Concurrency: goroutines & channels

channel: a type-safe generalization of Unix pipes

- inspired by Hoare's Communicating Sequential Processes (1978)

goroutine: a function executing concurrently with other goroutines in the same address space

- run multiple parallel computations simultaneously
- loosely like threads but much lighter weight

channels coordinate computations by explicit communication

- locks, semaphores, mutexes, etc., are much less often used

Goroutine

- It is a Go function or method
- It is executed concurrently in the same address space as other goroutines.
- A running program consists of one or more goroutines.

Goroutine



- Goroutines run in the same address space, so access to shared memory must be synchronized.
- The sync package provides useful primitives, although you won't need them much in Go as there are other primitives.

Goroutines vs, thread

- Goroutines are extremely cheap when compared to threads. Only few kb in stack size (stack can grow according to needs) Threads stack size is fixed.
- The Goroutines are multiplexed to fewer number of OS threads.
- Goroutines communicate using channels. Channels can be thought of as a pipe using which Goroutines communicate.

Goroutine

1. Goroutines are managed by the go runtime.
2. Goroutine are not hardware dependent.
3. Goroutines have easy communication medium known as channel.
4. Due to the presence of channel one goroutine can communicate with other goroutine with low latency
5. Goroutine does not have ID because go does not have Thread Local Storage.
6. Goroutines are cheaper than threads. The cost of threads are higher than goroutine.
7. They are cooperatively scheduled
8. They have faster startup time than threads.
9. Goroutine has growable segmented stacks.

Thread

1. Operating system threads are managed by kernel.
2. Thread are hardware dependent
3. Thread does not have easy communication medium.
4. Due to lack of easy communication medium inter-threads communicate takes place with high latency.
5. Threads have their own unique ID because they have Thread Local Storage.
6. The cost of threads are higher than goroutine.
7. They are preemptively scheduled.
8. They have slow startup time than goroutines.
9. Threads does not have growable segmented stacks.

Starting a goroutine: invoke a function or method via **go**

```
func IsReady(what string, minutes int64) {  
    time.Sleep(minutes * 60*1e9) // Unit is nanosecs.  
    fmt.Println(what, "is ready")  
}
```

```
go IsReady("tea", 6)  
go IsReady("coffee", 2)
```

```
fmt.Println("I'm waiting...")  
Prints:
```

I'm waiting... (right away)
coffee is ready (2 minutes later)
tea is ready (6 minutes later)

When a new Goroutine is started, the goroutine call returns immediately. Control does not wait for the Goroutine to finish executing. The main Goroutine should be running for any other Goroutines to run.

Another Example

```
package main
import ("fmt"; "time")
func say(s string) {
    for i := 0; i < 5; i++ {
        time.Sleep(100 * time.Millisecond)
        fmt.Println(s)
    }
}
func main() {
    go say("world")
    say("hello")
}
```

```
world
hello
hello
world
world
hello
hello
world
world
hello
```

Initialization



Program initialization runs in a single goroutine, but that goroutine may create other goroutines, which run concurrently.



If a package p imports package q , the completion of q 's init functions happens before the start of any of p 's.



The completion of all init functions is synchronized before the start of the function `main.main`.

Goroutine creation & destruction

- *The go statement that starts a new goroutine is synchronized before the start of the goroutine's execution.*

The exit of a goroutine is not guaranteed to be synchronized before any event in the program.

```
package main
import "fmt"
var a string

func hello() {
    go func() { a = "ciao" }()
    print(a)
}

func main() {
    fmt.Println("Hello World")
    go hello()
    fmt.Println(a)
}
```

If the effects of a goroutine must be observed by another goroutine, use a synchronization mechanism such as a lock or channel communication to establish a relative ordering.

Example: multiple goroutines

```
package main
```

```
import (  
    "fmt"  
    "time"  
)
```

```
func numbers() {  
    for i := 1; i <= 5; i++ {  
        time.Sleep(250 * time.Millisecond)  
        fmt.Printf("%d ", i)  
    }  
}
```

```
func alphabets() {  
    for i := 'a'; i <= 'e'; i++ {  
        time.Sleep(400 * time.Millisecond)  
        fmt.Printf("%c ", i)  
    }  
}
```

```
func main() {  
    go numbers()  
    go alphabets()  
    time.Sleep(3000 * time.Millisecond)  
    fmt.Println("main terminated")  
}
```

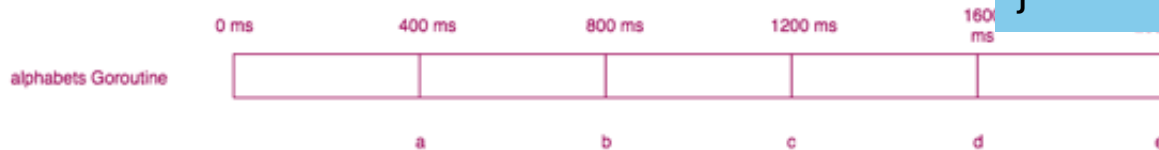
1 a 2 3 b 4 c 5 d e main terminated

=== Code Execution Successful ===

```
func numbers() {
    for i:= 1; i <= 5; i++ {
        time.Sleep(250 * time.Millisecond)
        fmt.Printf("%d ", i)
    }
}
```

```
func alphabets() {
    for i:= 'a'; i <= 'e'; i++ {
        time.Sleep(400 * time.Millisecond)
        fmt.Printf("%c ", i)
    }
}
```

```
func main() {
    go numbers()
    go alphabets()
    time.Sleep(3000 * time.Millisecond)
    fmt.Println("main terminated")
}
```



Anonymous Goroutine

In Go language, you can also start Goroutine for an anonymous function or in other words, you can create an anonymous Goroutine simply by using go keyword as a prefix of that function as shown in the below Syntax:

```
// Anonymous function call  
go func (parameter_list){  
    // statement  
}(arguments)
```

```
package main
```

```
import (  
    "fmt"  
    "time"  
)
```

```
// Main function  
func main() {
```

```
    fmt.Println("Welcome!! to Main function")  
    // Creating Anonymous Goroutine  
    go func() {  
        fmt.Println("Welcome!! to Goroutine")  
    }()  
    time.Sleep(1 * time.Second)  
    fmt.Println("GoodBye!! to Main function")  
}
```

```
Welcome!! to Main function  
Welcome!! to Goroutine  
GoodBye!! to Main function
```

```
=== Code Execution Successful ===
```


Channel



Communication: Unless two goroutines can communicate, they can't coordinate.



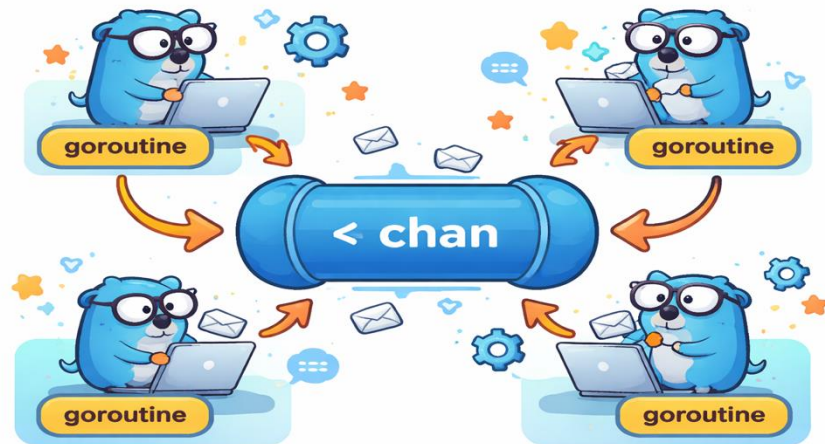
Type `chan`:. It also has special control structures that build on channels to make concurrent programming easy.



`Chan` type is referred to a base type.

CHANNEL

Channels are the pipes that connect concurrent goroutines. You can send values into channels from one goroutine and receive those values into another goroutine.



Channel declaring

- In its simplest form the type looks like this:

chan elementType

- With a value of this type, you can send and receive items of elementType.
- Channel is bidirectional
- Channels are a reference type,
 - if you assign one chan variable to another, both variables access the same channel
 - The zero value for a channel is nil
- It also means you use make to allocate one:

var ch = make(chan int)

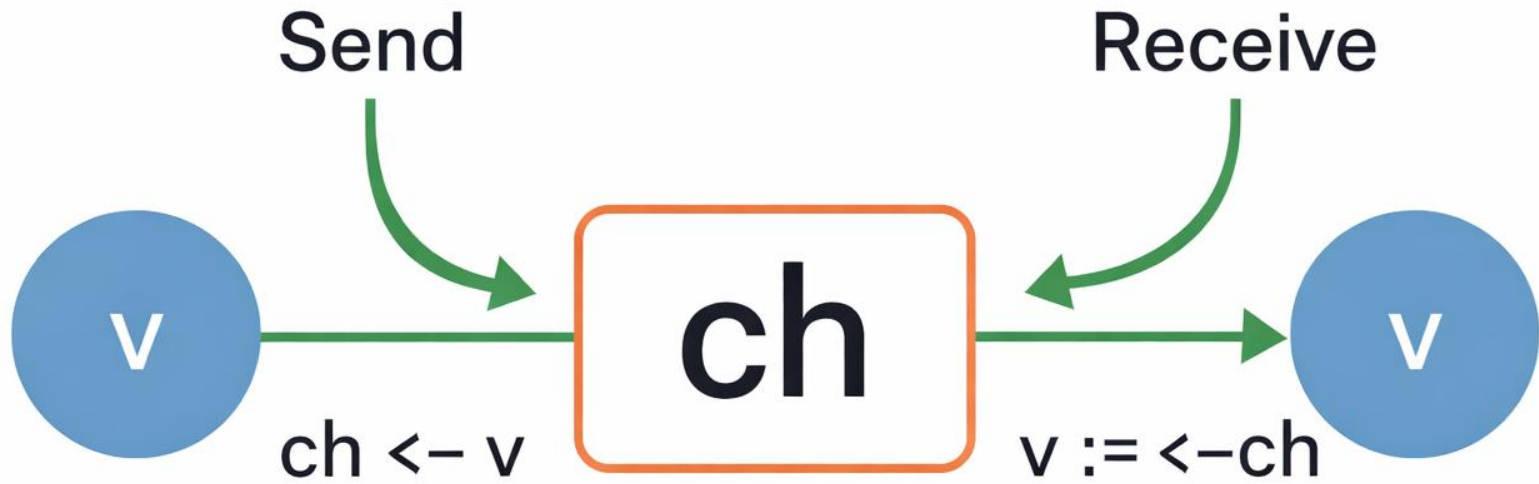
Example

channel a is nil, going to define it
Type of a is chan int
=== Code Execution Successful ===

```
package main

import "fmt"

func main() {
    var a chan int
    if a == nil {
        fmt.Println("channel a is nil, going to define it")
        a = make(chan int)
        fmt.Printf("Type of a is %T", a)
    }
}
```



Use channel:
Send &
receive

$ch \leftarrow v$ // Send v to channel ch .
 $v := \leftarrow ch$ // Receive from ch , and
// assign value to v .

```
package main
import (
    "fmt"
)
```

"chan" => in the function parameter, means that we will read and write from that channel, its a data type.
"chan<-" => means that we will write only
"<-chan" => means that we will read only

```
func createUser(name string, done chan<- bool) {
    fmt.Println(name + " has been created successfully")
    done <- true
}
func uploadUserImage(done chan<- bool) {
    fmt.Println("User image has been successfully uploaded to the cloud
!!")
    done <- true
}
```

```
func main() {

    done := make(chan bool)

    go uploadUserImage(done)
    fmt.Println("Image uploaded: ", <-done)

    go createUser("wassim", done)
    fmt.Println("User created: ", <-done)
}
```

User image has been successfully uploaded
to the cloud !!

Image uploaded: true

wassim has been created successfully

User created: true

=== Code Execution Successful ===



The communication operator

The arrow points in the direction of data flow.

As a binary operator, `<-` sends the value on the right to the channel on the left:

```
c := make(chan int)
```

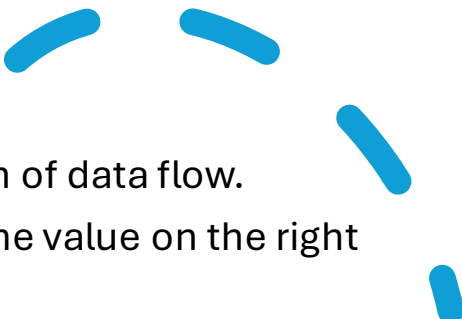
```
c <- 1           // send 1 on c (flowing into c)
```

As a prefix unary operator, `<-` receives from a channel:

```
v = <-c // receive value from c, assign to v
```

```
<-c      // receive value, throw it away
```

```
i := <-c // receive value, initialize i
```



Semantics: **Sends and receives are blocking**

By default, communication is synchronous.

- 1) A send operation on a channel blocks until a receiver is available for the same channel.
- 2) A receive operation for a channel blocks until a sender is available for the same channel.
- .

Communication is therefore a form of synchronization: two goroutines exchanging data through a channel synchronize at the moment of communication

```
func pump(ch chan int) {  
    for i := 0; ; i++ { ch <- i }  
}
```

```
func suck(ch chan int) {  
    for { fmt.Println(<-ch) }  
}
```

```
ch1 := make(chan int)  
go pump(ch1) // pump hangs; we run  
fmt.Println(<-ch1) // prints 0
```



```

func main() {
    ch1 := make(chan int)
    go pump(ch1)

    var v int
    for {
        v = <-ch1
        if v == 31415 {
            break
        }
    }

    fmt.Println(v) // Prints 31415

    go suck(ch1)

    //select {} // evita che main termini
}

```

31415

=== Code Execution Successful ===

Functions returning channels

In the previous example, pump was like a generator spewing out values. But there was a lot of fuss allocating channels etc. Let's package it up into a function returning the channel of values.

```
func pump() chan int {  
    ch := make(chan int)  
    go func() {  
        for i := 0; ; i++ { ch <- i }  
    }()  
    return ch  
}
```

```
stream := pump()
```

```
fmt.Println(<-stream)
```

```
// prints 0
```

Example

```
package main

import (
    "fmt"
)

func hello(done chan bool) {
    fmt.Println("Hello world goroutine")
    done <- true
}

func main() {
    done := make(chan bool)
    go hello(done)
    <-done
    fmt.Println("main function")
}
```

Example

-5 17 12

```
package main
import "fmt"
func sum(s []int, c chan int) {
    sum := 0
    for _, v := range s {
        sum += v
    }
    c <- sum // send sum to
```

```
c
}
```

```
func main() {
    s := []int{7, 2, 8, -9, 4, 0}

    c := make(chan int)
    go sum(s[:len(s)/2], c)
    go sum(s[len(s)/2:], c)
    x, y := <-c, <-c // receive from c

    fmt.Println(x, y, x+y)
}
```

Buffered Channels

- Channels can be buffered. Provide the buffer length as the second argument to `make` to initialize a buffered channel:
`ch := make(chan int, 100)`
- Sends to a buffered channel block only when the buffer is full. Receives block when the buffer is empty.

Buffered Channels

1
2

- Channels can be buffered. Provide the buffer length as the second argument to `make` to initialize a buffered channel:
`ch := make(chan int, 100)`
- Sends to a buffered channel block only when the buffer is full. Receives block when the buffer is empty.

```
package main

import "fmt"

func main() {
    ch := make(chan int, 2)
    ch <- 1
    ch <- 2
    fmt.Println(<-ch)
    fmt.Println(<-ch)
}
```

Buffer

Buffer is not part of the type

Note that the buffer's size, or even its existence, is not part of the channel's type, only of the value.

This code is therefore legal, although dangerous:

```
buf = make(chan int, 1)
unbuf = make(chan int)
buf = unbuf
unbuf = buf
```

Buffering is a property of the value, not of the type.

Closing a channel

How does range know when the channel is done delivering data?

The sender calls the built-in function close:

```
close(ch)
```

Receiver tests if the sender has closed the channel using "comma ok":

```
val, ok := <-ch
```

Result is (value, true) while values are available; once channel is closed and drained, result is (zero, false).

Range and Close

- A sender can close a channel to indicate that no more values will be sent. Receivers can test whether a channel has been closed by assigning a second parameter to the receive expression: after
 `v, ok := <-ch`
- `ok` is `false` if there are no more values to receive and the channel is closed.
- The loop `for i := range c` receives values from the channel repeatedly until it is closed.

Range and Close

- A sender can close a channel to indicate that no more values will be sent. Receivers can test whether a channel has been closed by assigning a second parameter to the receive expression: after `v, ok := <-ch`
- `ok` is false if there are no more values to receive and the channel is closed.
- The loop `for i := range c` receives values from the channel repeatedly until it is closed.

Only the sender should close a channel, never the receiver. Sending on a closed channel will cause a panic. Channels aren't like files; you don't usually need to close them. Closing is only necessary when the receiver must be told there are no more values coming, such as to terminate a range loop.

```
package main
import ("fmt")
func fibonacci(n int, c chan int) {
    x, y := 0, 1
    for i := 0; i < n; i++ {
        c <- x
        x, y = y, x+y
    }
    close(c)
}
func main() {
    c := make(chan int, 10)
    go fibonacci(cap(c), c)
    for i := range c {
        fmt.Println(i)
    }
}
```

0
1
1
2
3
5

Range and channels

- The range clause on for loops accepts a channel as an operand, in which case the for loops over the values received from the channel.
- We rewrote pump; here's the rewrite for suck, making it launch the goroutine as well:

```
func suck(ch chan int) {  
    go func() {  
        for v := range ch {  
            fmt.Println(v)  
        }()  
    }  
    suck(pump()) // doesn't block now
```

Range on a channel, by hand

A for range on a channel such as:

```
for value := range <-ch {  
    use(value)  
}
```

is equivalent to:

```
for {  
    value, ok := <-ch  
    if !ok {  
        break  
    }  
    use(value)  
}
```

Channel directionality

All channels are created bidirectional, but we can assign them to directional channel variables. A channel type may be annotated to specify that it may only send or only receive:

```
var recvOnly <-chan int  
var sendOnly chan<- int
```

Useful for instance in functions, for (type) safety:

```
func sink(ch <-chan int) {  
    for { <-ch }  
}  
func source(ch chan<- int) {  
    for { ch <-1 }  
}
```

```
c := make(chan int) // bidirectional  
go source(c)  
go sink(c)
```

Synchronous channels

Synchronous channels are unbuffered.

Sends do not complete until a receiver has accepted the value.

`c`

```
:= make(chan int)
go func() {
    time.Sleep(60*1e9)
    x := <-c
    fmt.Println("received", x)
}()
fmt.Println("sending", 10)
C <-10
fmt.Println("sent", 10)
```

Output:

```
sending 10 (happens immediately)
sent 10 (60s later, these 2 lines appear)
received 10
```

Length vs Capacity

- The capacity of a buffered channel is the number of values that the channel can hold. This is the value we specify when creating the buffered channel using the `make` function.
- The length of the buffered channel is the number of elements currently queued in it.


```
package main
```

```
import (  
    "fmt"  
)
```

```
func main() {  
    ch := make(chan string, 3)  
    ch <- " Prima stringa"  
    ch <- " Seconda stringa"  
    fmt.Println("capacity is", cap(ch))  
    fmt.Println("length is", len(ch))  
    fmt.Println("read value", <-ch)  
    fmt.Println("new length is", len(ch))  
}
```

```
capacity is 3  
length is 2  
read value Prima stringa  
new length is 1
```

SELECT

- Select Statement in Go Language is just like switch statement, but in the select statement, case statement refers to communication, i.e. sent or receive operation on the channel.

```
select{  
  
    case SendOrReceive1: // Statement  
    case SendOrReceive2: // Statement  
    case SendOrReceive3: // Statement  
    .....  
    default: // Statement  
}
```

Select

1. Select statement waits until the communication(send or receive operation) is prepared for some cases to begin
2. If a select statement does not contain any case statement, then that select statement waits forever.
3. The default statement is used to protect select statement from blocking. This statement executes when there is no case statement is ready to proceed.
4. The blocking of select statement means when there is no case statement is ready and the select statement does not contain any default statement, then the select statement block until at least one case statement or communication can proceed.
5. In select statement, if multiple cases are ready to proceed, then one of them can be selected randomly.

Select

The select statement lets a goroutine wait on multiple communication operations.

Each case must be a communication, either send or receive.

A select blocks until one of its cases can run, then it executes that case. It chooses one at random if multiple are ready.

Select executes one runnable case at random. If no case is runnable, it blocks until one is. A default clause is always runnable.

```
ci, cs := make(chan int), make(chan string)
select {
    case v := <-ci:
        fmt.Printf("received %d from ci\n", v)
    case v := <-cs:
        fmt.Printf("received %s from cs\n", v)
}
```

```

package main
import "fmt"
func fibonacci(c, quit chan int) {
    x, y := 0, 1
    for {
        select {
        case c <- x: x, y = y, x+y
        case <-quit: fmt.Println("quit")
        return
        }
    }
}

```

```

func main() {
    c := make(chan int)
    quit := make(chan int)
    go func() {
        for i := 0; i < 10; i++ {
            fmt.Println(<-c)
        }
    }()
    quit <- 0
    fibonacci(c, quit)
}

```

```

0
1
1
2
3
5
8
13
21
34
quit

```

Default Selection

The default case in a select is run if no other case is ready.

Use a default case to try a send or receive without blocking:

```
select {  
  case i := <-c:  
    // use i  
  default:  
    // receiving from c would block  
}
```

PACKAGE time

```
func After(d Duration) <-chan Time
```

After waits for the duration to elapse and then sends the current time on the returned channel. It is equivalent to `NewTimer(d).C`.

The underlying `Timer` is not recovered by the garbage collector until the timer fires. If efficiency is a concern, use `NewTimer` instead and call `Timer.Stop` if the timer is no longer needed.

```
func Tick(d Duration) <-chan Time
```

Tick is a convenience wrapper for NewTicker providing access to the ticking channel only.

While Tick is useful for clients that have no need to shut down the Ticker, be aware that without a way to shut it down the underlying Ticker cannot be recovered by the garbage collector; it "leaks". Unlike NewTicker, Tick will return nil if $d \leq 0$.

```
func Sleep(d Duration)
```

Sleep pauses the current goroutine for at least the duration d . A negative or zero duration causes Sleep to return immediately.


```
type Duration int64
```

```
type Location struct {  
    // contains filtered or unexported  
    fields  
}
```

```
package main
```

```
import ("fmt"; "time")
```

```
func main() {
```

```
    tick := time.Tick(100 * time.Millisecond)
```

```
    boom := time.After(500 * time.Millisecond)
```

```
    for {
```

```
        select {
```

```
        case <-tick:
```

```
            fmt.Println("tick.")
```

```
        case <-boom:
```

```
            fmt.Println("BOOM!")
```

```
            return
```

```
        default:
```

```
            fmt.Println("  .")
```

```
            time.Sleep(50 *
```

```
time.Millisecond)
```

```
        }
```

```
    }
```

```
}
```

tick.

```
func Tick(d Duration) <-chan Time
```

Package time: After

Can a communication succeed in a given interval?

Package time contains the After function:

```
func After(ns int64) <-chan int64
```

It delivers a value (the current time) on the returned channel after the specified interval.

Use it in select to implement timeout:

```
select {
    case v := <-ch:
        fmt.Println("received", v)
    case <-time.After(30*1e9):
        fmt.Println("timed out after 30 seconds")
}
```

```
func After(d Duration) <-chan Time
```

Package Sync

Tipo	Scopo principale
Mutex	Accesso esclusivo a una risorsa
RWMutex	Lettura concorrente, scrittura esclusiva
WaitGroup	Aspettare che goroutine completino lavoro
Once	Eseguire una funzione una sola volta
Cond	Sincronizzazione basata su segnali
Map	Mappa concorrente sicura

Package Sync: primitives

Package sync provides basic synchronization primitives such as mutual exclusion locks.

Other than the `Once` and `WaitGroup` types, most are intended for use by low-level library routines.

Higher-level synchronization is better done via channels and communication.

A `Locker` represents an object that can be locked and unlocked.

```
type Locker interface {  
    Lock()  
    Unlock()  
}
```

```
var rw sync.RWMutex
rw.RLock()
// lettura concorrente
rw.RUnlock()
rw.Lock()
// scrittura esclusiva
rw.Unlock()
```

WaitGroup

- *To wait for multiple goroutines to finish, we can use a wait group.*

```
package main                                var wg sync.WaitGroup

import (                                     func createUser(name string) {
    "fmt"                                   defer wg.Done()
    "sync"                                   fmt.Println(name + " has been created successfully")
    "time"                                   }
)

func uploadUserImage() {
    defer wg.Done()
    time.Sleep(time.Second * 3)
    fmt.Println("User image uploaded to the cloud
!!")
}

func main() {
    go createUser("wassim")
    go uploadUserImage()

    wg.Add(2)
    wg.Wait()
}
```


type Cond

Cond implements a condition variable, a rendezvous point for goroutines waiting for or announcing the occurrence of an event.

Each Cond has an associated Locker L (often a *Mutex or *RWMutex), which must be held when changing the condition and when calling the Wait method.

A Cond must not be copied after first use.

```
type Cond struct {  
    // L is held while observing or changing the condition  
    L Locker  
    // contains filtered or unexported fields  
}
```

```
package main
```

```
import (  
    "fmt"  
    "sync"  
)
```

```
func main() {  
    var mu sync.Mutex  
    cond := sync.NewCond(&mu)  
    ready := false
```

```
    go func() {  
        mu.Lock()  
        for !ready {  
            cond.Wait() // aspetta il segnale  
        }  
        fmt.Println("Goroutine svegliata!")  
        mu.Unlock()  
    }()
```

```
    mu.Lock()
```

```
    ready = true
```

```
    cond.Signal() // segnala una goroutine in attesa
```

```
    mu.Unlock()
```

```
    // Attesa per vedere l'output (solo demo)
```

```
    var input string
```

```
    fmt.Scanln(&input)
```

```
}
```

```
package main
```

```
import (
```

```
    "fmt"
```

```
    "math/r
```

```
and"
```

```
    "sync"
```

```
    "time"
```

```
)
```

```
// This function must be called when cond.L is locked.
```

```
    checkCondition := func() bool {
```

```
        fmt.Println(values)
```

```
        for i := 0; i < N; i++ {
```

```
            if values[i] == "" {
```

```
                return false
```

```
            }
```

```
        }
```

```
        return true
```

```
    }
```

```
    cond.L.Lock()
```

```
    defer cond.L.Unlock()
```

```
    for !checkCondition() {
```

```
        // Must be called when cond.L is locked.
```

```
        cond.Wait()
```

```
    }
```

```
}
```

```

func main() {
    rand.Seed(time.Now().UnixNano())
    const N = 10
    var values [N]string
    cond := sync.NewCond(&sync.Mutex{})
    for i := 0; i < N; i++ {
        d := time.Second * time.Duration(rand.Intn(100))
        go func(i int) {
            time.Sleep(d) // simulate a workload
            // Changes must be made when cond.L lock is locked.
            cond.L.Lock()
            values[i] = string('a' + i)
            // Notify when cond.L lock is locked.
            cond.Broadcast()
            cond.L.Unlock()
            // "cond.Broadcast()" can also be put here, when
            // unlocked.
            //cond.Broadcast()
        }(i)
    }
}

```

```

[   ]
[  g ]
[ e g ]
[ e g j]
[ e g h j]
[ b e g h j]
[ b c e g h j]
[ b c e f g h j]
[ b c d e f g h j]
[ a b c d e f g h j]
[ a b c d e f g h i j]

```

Once Type

- A `*sync.Once` value has a `Do(f func())` method, which takes a solo parameter with type `func()`.
- For an addressable `Once` value `o`, the method call `o.Do()` , can be concurrently executed multiple times, in multiple goroutines.
- The arguments of these `o.Do()` calls should (but are not required to) be the same function value.
- Among these `o.Do` method calls, only exact one argument function will be invoked.
- The invoked argument function is guaranteed to exit before any `o.Do` method call returns.
- The code in the invoked argument function is guaranteed to be executed before any `o.Do` method call returns.
- Generally, a `Once` value is used to ensure that a piece of code will be executed exactly once in concurrent programming.

```
package main
```

```
import (  
    "log"  
    "sync"  
)
```

Hello will be
printed once,
but world! will be
printed five
times.

And Hello is
guaranteed to be
printed before all
five world!

```
func main() {  
    log.SetFlags(0)  
    x := 0  
    doSomething := func() {  
        x++  
        log.Println("Hello")  
    }  
    var wg sync.WaitGroup  
    var once sync.Once  
    for i := 0; i < 5; i++ {  
        wg.Add(1)  
        go func() {  
            defer wg.Done()  
            once.Do(doSomething)  
            log.Println("world!")  
        }()  
    }  
    wg.Wait()  
    log.Println("x =", x) // x = 1  
}
```

type Mutex

A Mutex is a mutual exclusion lock. The zero value for a Mutex is an unlocked mutex.

A Mutex must not be copied after first use.

```
type Mutex struct {  
    // contains filtered or unexported fields  
}
```

Go's standard library provides mutual exclusion with `sync.Mutex` and its two methods:

- Lock
- Unlock

We can define a block of code to be executed in mutual exclusion by surrounding it with a call to Lock and Unlock as shown on the Inc method.

We can also use defer to ensure the mutex will be unlocked as in the Value method.

LOCK

```
package main

import (
    "fmt"
    "sync"
    "time"
)

func main() {

    var mu sync.Mutex          // Crea un mutex
    var counter int            // Crea una variabile
                                condivisa
    // Crea due goroutine che incrementano la variabile condivisa
    go func() {
        for i := 0; i < 1000; i++ {
            // Acquisisce il lock
            mu.Lock()
            counter++           // Incrementa la
                                variabile condivisa
            mu.Unlock()         // Rilascia il lock
        }
    }()

    // Attendi che le goroutine finiscano
    time.Sleep(time.Second)
    // Stampa il valore della variabile condivisa
    fmt.Println(counter)
}
```



```

import (
    "fmt"
    "sync"
    "time"
)

// SafeCounter is safe to use concurrently.
type SafeCounter struct {
    v  map[string]int
    mux sync.Mutex
}

// Inc increments the counter for the given
key.
func (c *SafeCounter) Inc(key string) {
    c.mux.Lock()
    // Lock so only one goroutine at a
time can access the map c.v.
    c.v[key]++
    c.mux.Unlock()
}

/

```

1000

```

/ Value returns the current value of the
counter for the given key.
func (c *SafeCounter) Value(key string) int {
    c.mux.Lock()
    // Lock so only one goroutine at a
time can access the map c.v.
    defer c.mux.Unlock()
    return c.v[key]
}

func main() {
    c := SafeCounter{v:
make(map[string]int)}
    for i := 0; i < 1000; i++ {
        go c.Inc("somekey")
    }

    time.Sleep(time.Second)
    fmt.Println(c.Value("somekey"))
}

```

Multiplexing server

Channels are first-class values, which means they can be sent over channels.

This property makes it easy to write a service multiplexer since the client can supply, along with its request, the channel on which to reply.

```
chanOfChans := make(chan chan int)
```

Or more typically

```
type Reply struct { ... }
type Request struct {
    arg1, arg2 someType
    replyc chan *Reply
}
```

Multiplexing server

```
type request struct {  
    a, b int  
    replyc chan int  
}  
type binOp func(a, b int) int  
func run(op binOp, req *request) {  
    req.replyc <- op(req.a, req.b)  
}  
func server(op binOp, service <-chan *request) {  
    for {  
        req := <-service // requests arrive here  
        go run(op, req) // don't wait for op  
    }  
}
```

Starting the server

Use the "function returning channel" idiom to create a channel to a new server:

```
func startServer(op binOp) chan<- *request {  
    service := make(chan *request)  
    go server(op, req)  
    return service  
}  
adderChan := startServer(func(a, b int) int { return a +b })
```

The client

```
func (r *request) String() string {  
    return fmt.Sprintf("%d+%d=%d",r.a, r.b, <-r.replyc)  
}
```

```
req1 := &request{7, 8, make(chan int)}
```

```
req2 := &request{17, 18, make(chan int)}
```

- Requests are ready; send them:

```
adderChan <- req1
```

```
adderChan <- req2
```

- Can retrieve results in any order; r.replyc demuxes:

```
fmt.Println(req2, req1)
```

Starting the server

- The rest of the code is mostly the same, with an extra channel:

```
func startServer(op binOp) (service chan<- *request,  
                           quit chan<- bool) {  
    service = make(chan *request)  
    quit = make(chan bool)  
    go server(op, service, quit)  
    return service, quit  
}  
adderChan, quitChan := startServer(  
    func(a, b int) int { return a + b })
```

Teardown: the client

- The client is unaffected until it's ready to shut down the server:

```
req1 := &request{7, 8, make(chan int)}  
req2 := &request{17, 18, make(chan int)}  
adderChan <- req1  
adderChan <- req2  
fmt.Println(req2, req1)
```

- All done, signal server to exit:

```
quitChan <- true
```

Chaining

```
package main
import ("flag"; "fmt")
var nGoroutine = flag.Int("n", 100000, "how many")
func f(left, right chan int) { left <- 1 + <-right }
func main() {
    flag.Parse()
    leftmost := make(chan int)
    var left, right chan int = nil, leftmost
    for i := 0; i < *nGoroutine; i++ {
        left, right = right, make(chan int)
!        go f(left, right)
    }
    right <- 0 // bang!
    x := <-leftmost // wait for completion
    fmt.Println(x) // 100000
}
```


Example: Channel as Buffer cache

```
var freeList = make(chan *Buffer, 100)
var serverChan = make(chan *Buffer)
func server() {
    for {
        b := <-serverChan // Wait for work
        process(b) // Process in buffer.
        select {
            case freeList <- b: // Reuse
            default: // drop it.
        }
    }
}
```

```
func client() {
    for {
        var b *Buffer
        select {
            case b = <-freeList://Grab one if available.
            default: b = new(Buffer) // Allocate if not.
        }
        load(b) // Read next request into b.
        serverChan <- b // Send request to server.
    }
}
```

Teardown

In the multiplex example, the server runs forever.

To tear it down cleanly, signal with a channel. This server has the same functionality but with a quit channel:

```
func server(op binOp, service <-chan *request, quit <-chan bool) {  
    for {  
        select {  
            case req := <-service:    go run(op, req)  
                                        // don't wait for it  
            case <-quit: return  
        }  
    }  
}
```

Worker Pool Implementation

```
package main
```

```
import ("fmt"  
        "time"  
        "sync"  
        "math/rand"  
)
```

```
type Job struct {  
    id    int  
    randomno int  
}
```

```
type Result struct {  
    job      Job  
    sumofdigits int  
}
```

```
var jobs = make(chan Job, 10)  
var results = make(chan Result, 10)
```

```
func digits(number int) int {  
    sum := 0  
    no := number  
    for no != 0 {  
        digit := no % 10  
        sum += digit  
        no /= 10  
    }  
    time.Sleep(2 * time.Second)  
    return sum  
}
```

```
func worker(wg *sync.WaitGroup) {  
    for job := range jobs {  
        output := Result{job,  
            digits(job.randomno)}  
        results <- output  
    }  
    wg.Done()  
}
```

```

func createWorkerPool(noOfWorkers int)
{
    var wg sync.WaitGroup
    for i := 0; i < noOfWorkers; i++ {
        wg.Add(1)
        go worker(&wg)
    }
    wg.Wait()
    close(results)
}

```

```

func result(done chan bool) {
    for result := range results {
        fmt.Printf("Job id %d, input random no %d , sum of digits %d\n",
            result.job.id, result.job.randomno, result.sumofdigits)
    }
    done <- true
}

```

```

func allocate(noOfJobs int) {
    for i := 0; i < noOfJobs; i++ {
        randomno := rand.Intn(999)
        job := Job{i, randomno}
        jobs <- job
    }
    close(jobs)
}

```

```
func main() {  
    startTime := time.Now()  
    noOfJobs := 100  
    go allocate(noOfJobs)  
    done := make(chan bool)  
    go result(done)  
    noOfWorkers := 10  
    createWorkerPool(noOfWorkers)  
    <-done  
    endTime := time.Now()  
    diff := endTime.Sub(startTime)  
    fmt.Println("total time taken ", diff.Seconds(), "seconds")  
}
```

UNSAFE PACKAGE

- The built-in package `unsafe`, known to the compiler and accessible through the import path `"unsafe"`, provides facilities for low-level programming including operations that violate the type system.
- A package using `unsafe` must be vetted manually for type safety and may not be portable.

UNSAFE PACKAGE INTERFACE

package unsafe

- type ArbitraryType int // shorthand for an arbitrary Go type; it is not a real type
- type Pointer *ArbitraryType
- func Alignof(variable ArbitraryType) uintptr
- func Offsetof(selector ArbitraryType) uintptr
- func Sizeof(variable ArbitraryType) uintptr
- type IntegerType int // shorthand for an integer type; it is not a real type
- func Add(ptr Pointer, len IntegerType) Pointer
- func Slice(ptr *ArbitraryType, len IntegerType) []ArbitraryType

- A Pointer is a pointer type but a Pointer value may not be dereferenced.
- Any pointer or value of underlying type uintptr can be converted to a type of underlying type Pointer and vice versa.
- The effect of converting between Pointer and uintptr is implementation-defined.

Web Crawler

- We use Go's concurrency features to parallelize a web crawler.
- Modify the Crawl function to fetch URLs in parallel without fetching the same URL twice.

Version 1: no parallelism

```
func main() {  
    start := time.Now()  
    for _, site := range os.Args[1:] {  
        count("http://" + site)  
    }  
    fmt.Printf("%.2fs total\n",  
time.Since(start).Seconds())  
}
```

```
func count(url string) {  
    start := time.Now()  
    r, err := http.Get(url)  
    if err != nil {  
        fmt.Printf("%s: %s\n", url, err)  
        return  
    }  
    n, _ := io.Copy(ioutil.Discard, r.Body)  
    r.Body.Close()  
    dt := time.Since(start).Seconds()  
    fmt.Printf("%s %d [%.2fs]\n", url, n, dt)  
}
```

Version 2: with parallelism

```
func main() {  
    start := time.Now()  
    c := make(chan string)  
    n := 0  
    for _, site := range os.Args[1:] {  
        n++  
        go count("http://" + site, c)  
    }  
    for i := 0; i < n; i++ {  
        fmt.Print(<-c)  
    }  
    fmt.Printf("%.2fs total\n",  
time.Since(start).Seconds())  
}
```

```
func count(url string, c chan<- string) {  
    start := time.Now()  
    r, err := http.Get(url)  
    if err != nil {  
        c <- fmt.Sprintf("%s: %s\n", url, err)  
        return  
    }  
    n, _ := io.Copy(ioutil.Discard, r.Body)  
    r.Body.Close()  
    dt := time.Since(start).Seconds()  
    c <- fmt.Sprintf("%s %d [%.2fs]\n", url, n,  
dt)  
}
```

Package http

- Package http provides HTTP client and server implementations.

```
import "net/http"
```

- Get, Head, Post, and PostForm make HTTP (or HTTPS) requests:

```
resp, err := http.Get("http://example.com/")
```

```
...
```

```
resp, err := http.Post("http://example.com/upload", "image/jpeg", &buf)
```

```
...
```

```
resp, err := http.PostForm("http://example.com/form",  
    url.Values{"key": {"Value"}, "id": {"123"}})
```

Reflection

Reflect package

The first law of reflection: Reflection goes from interface value to reflection object.

- At the basic level, reflection is just a mechanism to examine the type and value pair stored inside an interface variable.
- in package reflect: Type and Value.

The second law of reflection

- Reflection goes from reflection object to interface value.
- Like physical reflection, reflection in Go generates its own inverse.

```
package main
```

```
import (  
    "fmt"  
    "reflect"  
)
```

```
type order struct {  
    ordId    int  
    customerId int  
}
```

```
func createQuery(q interface{}) {  
    t := reflect.TypeOf(q)  
    v := reflect.ValueOf(q)  
    fmt.Println("Type ", t)  
    fmt.Println("Value ", v)  
  
}
```

```
func main() {  
    o := order{  
        ordId:    456,  
        customerId: 56,  
    }  
    createQuery(o)  
}
```

reflect package

- Package reflect implements run-time reflection, allowing a program to manipulate objects with arbitrary types.
- The typical use is to take a value with static type `interface{}` and extract its dynamic type information by calling `TypeOf`, which returns a `Type`.
- A call to `ValueOf` returns a `Value` representing the run-time data.
- `Zero` takes a `Type` and returns a `Value` representing a zero value for that type.
- There are two functions `reflect.TypeOf()` and `reflect.ValueOf()` which return the `reflect.Type` and `reflect.Value` respectively. These two types are the base to create our query generator.

PACKAGE REFLECT FUNCTION

`func Copy(dst, src Value) int`

- Copy copies the contents of src into dst until either dst has been filled or src has been exhausted. It returns the number of elements copied. Dst and src each must have kind Slice or Array, and dst and src must have the same element type.
- As a special case, src can have kind String if the element type of dst is kind Uint8.

`func Swapper(slice interface{}) func(i, j int)`

- Swapper returns a function that swaps the elements in the provided slice.
- Swapper panics if the provided interface is not a slice.

PACKAGE REFLECT FUNCTION

`func DeepEqual(x, y interface{}) bool`

- DeepEqual reports whether x and y are “deeply equal,” defined as follows. Two values of identical type are deeply equal if one of the following cases applies. Values of distinct types are never deeply equal.
- Array → when their corresponding elements are deeply equal.
- Struct → if their corresponding fields, both exported and unexported, are deeply equal.
- Func → if both are nil; otherwise they are not deeply equal.
- Interface → if they hold deeply equal concrete values.
- Map → when all of the following are true: they are both nil or both non-nil, they have the same length, and either they are the same map object or their corresponding keys map to deeply equal values.
- Pointer → if they are equal using Go's == operator or if they point to deeply equal values